

LEO Satellite Energy Modeling

Linear Power Consumption & Hybrid System Architecture for Simulation Platforms

Model Selection Strategy

Evaluating three candidates for container-based energy emulation

Evaluating Energy Models



Model 1: RF/EIRP

Rejected: Requires radio parameters (beam count, gain) not modeled in OpenSN.



Model 2: DVFS

Rejected: Docker isolates physical clock speeds (\$f\$); inaccessible from container.



Model 3: Linear CPU

Selected: Utilizes measurable CPU load reporting via Linux cgroups.

| Selected Model: Linear CPU

100%

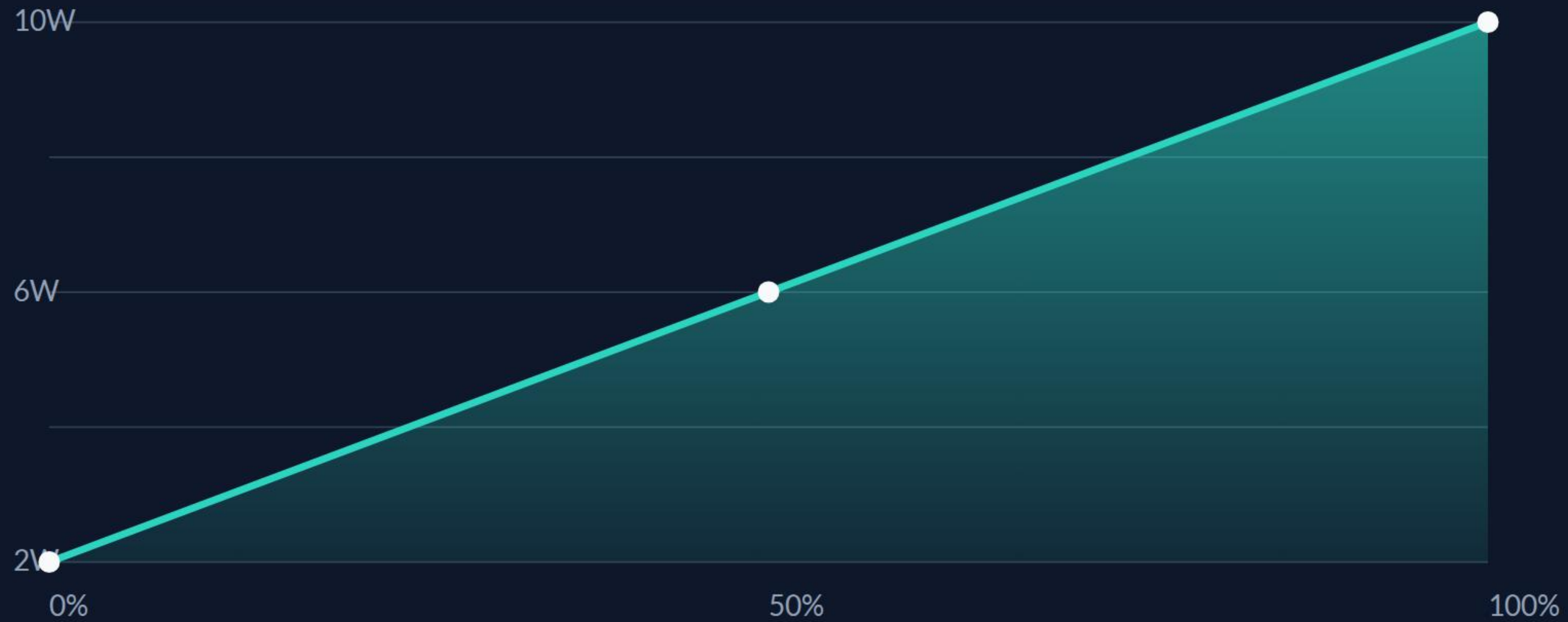
Implementable via Docker

Physics-Based Estimation

The Linear CPU model represents CMOS digital logic behavior. It uses standard "Idle + Activity" scaling, making it the only candidate compatible with container-based isolation protocols.

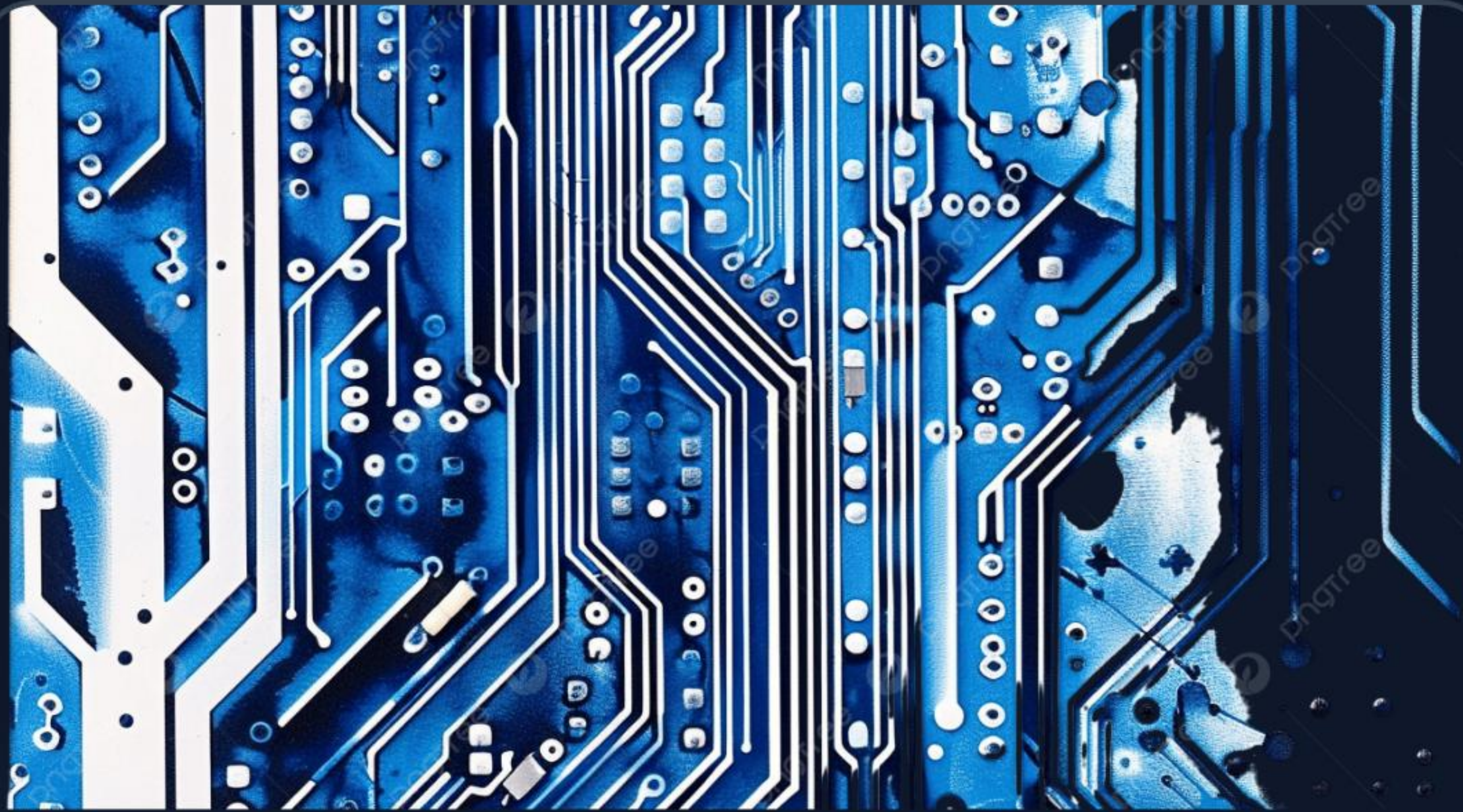
$$P_{\text{consume}}(t) = P_{\text{idle}} + (P_{\text{max}} - P_{\text{idle}}) \times u(t)$$

| Consumption vs. Utilization



Simulation data illustrates the Millivatt-to-Watt scaling typical for onboard satellite compute, transitioning linearly from Idle (P_{idle}) to Peak (P_{max}) load.

Measuring CPU Load: $u(t)$



Resource Utilization Proxy

The variable $u(t)$ is the CPU utilization fraction (0 to 1). In a Docker environment, this is measured relative to the container's allocated `nano_cpu` limit, not the physical host's absolute capacity.

- Read from `cgroup v2` system files.
- Calculated over a delta interval (Δt).
- Acts as a high-fidelity proxy for activity.

Model Comparison Summary

Candidate Model	Core Input	Feasibility	Reason for Reject/Select
RF / EIRP	Radio Band/Gains	Infeasible	OpenSN has no RF/Antenna concept.
DVFS Scaling	Clock Speed (\$f\$)	Infeasible	Docker hides physical clock frequency.
Linear CPU	Utilization %	Feasible	Exposed via Linux cgroups.

Architectural Design Choice

Balancing Centralized Coordination with Local Autonomy

The Hybrid Design Pattern

To optimize performance and realism, the energy module is split between the central controller and local satellite nodes.

1. Central Generation (P_{gen})

The Topology Controller calculates orbit physics and solar exposure.

2. Local Consumption (P_{cons})

Each container measures its own CPU load and updates its local battery State of Charge (SoC).

Data Flow & Implementation



Shared Config

main.py writes P_gen values to
/share/topo/ JSON files
every cycle.



Local Watcher

The satellite watcher process
reads usage from `cpu.stat`
internally.



Battery Sync

Local state (SoC) is updated
every 1s based on (P_gen -
P_cons) delta.